Project 2

A Numerical Solution to a Zero-Pressure Gradient Laminar Boundary Layer Equation

ME 5311

John Gomes

Professor Xinyu Zhao

April 2026

Introduction

The goal of this project is to obtain a numerical solution to the zero-pressure gradient laminar boundary layer equations. To solve this numerically, the Crank-Nicolson scheme, along with a central difference method will be used to solve following equations for continuity and boundary layer momentum.

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0 \tag{1}$$

$$u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} = \frac{\partial}{\partial y}\left(\nu\frac{\partial u}{\partial y}\right) \tag{2}$$

At the initial location, the velocity profile in the x direction is assumed to be the following piecewise function:

$$\frac{u}{U_\infty} = \begin{cases} \frac{y}{\delta}\left(\frac{3}{2} - \frac{1}{2}\left(\frac{y}{\delta}\right)^2\right) & \frac{y}{\delta} \leq 1 \\ 1 & \frac{y}{\delta} > 1 \end{cases} \tag{3}$$

For this, the following scales will be used:

$\nu_\infty = 1.5e10^{-6}\,\frac{m^2}{s}$, $U_\infty = 30\,\frac{m}{s}$, $L = 1.0\,m$, $\delta = 0.005\,m$, $y_{max} = 10\delta$, 150 y-grid points

Numerical Method Description

To solve this boundary layer problem numerically, we will use a coordinate transformation, allowing us to use a stretched grid in the y-direction. This will allow for higher accuracy closer to the wall. To do this, the physical coordinate, y, is related to the numerical domain coordinate, η, by the following. Note that the stretching factor, s, is equal to 10 for this experiment.

$$y = y_{max}\frac{\sinh(s\eta)}{\sinh(s)} \tag{4}$$

To correctly map into the η coordinate, the governing equation for momentum must be transformed.

$$u\frac{\partial u}{\partial x} + v\left(\frac{\partial u}{\partial \eta}\frac{\partial \eta}{\partial y}\right) = \nu\left(\frac{\partial^2 u}{\partial \eta^2}\left(\frac{\partial \eta}{\partial y}\right)^2 + \frac{\partial u}{\partial \eta}\frac{\partial^2 \eta}{\partial y^2}\right) \tag{5}$$

We must solve the first and second derivatives of η with respect to y. These solutions were found using a symbolic solver in Python (see end of code).

$$\frac{\partial \eta}{\partial y} = \frac{\sinh\,(s)}{s y_{max}\cosh(\eta s)} \tag{6}$$

$$\frac{\partial^2 \eta}{\partial y^2} = -\frac{\sinh^2(s)\sinh(\eta s)}{s y_{max}^2 \cosh^3(\eta s)} \tag{7}$$

Spatial discretization of the transformed momentum equation (eq. 5) is solved using a 4th order central difference method. The first and second derivative of u are discretized with the following equations.

$$\left(\frac{\partial u}{\partial \eta}\right)_i \approx \frac{-u_{i+2} + 8u_{i+1} - 8u_{i-1} + u_{i-2}}{12\Delta\eta} \tag{8}$$

$$\left(\frac{\partial^2 u}{\partial \eta^2}\right)_i \approx \frac{-u_{i+2} + 16u_{i+1} - 30u_i + 16u_{i-1} - u_{i-2}}{12\Delta\eta^2} \tag{9}$$

Note that these equations use a total of four neighbors, therefore, for boundary-adjacent nodes, a second-order scheme is used.

$$\left(\frac{\partial u}{\partial \eta}\right)_i \approx \frac{u_{i+1} - u_{i-1}}{2\Delta\eta} \tag{10}$$

$$\left(\frac{\partial^2 u}{\partial \eta^2}\right)_i \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta\eta^2} \tag{11}$$

Lastly, the Crank-Nicolson scheme is used to step in the x-direction. This is a first-order difference between the current and $n+1$ step, which is embedded within the C-N scheme, thus making it second-order accurate in the x direction.

$$\left(\frac{\partial u}{\partial x}\right)_i \approx \frac{u_i^{n+1} - u_i^n}{\Delta x} \tag{12}$$

The discretized solution can then be solved using Lower-Upper (LU) decomposition. The Crank-Nicolson solution shown above is implicit, meaning the solution of the future step is dependent on the current solution step.

$$Au^{n+1} = b \tag{13}$$

Here, A is the coefficient matrix, representing the $n+1$ step, while b represents all n^{th} step values, all currently known values.

After each step, the y-velocity, v is updated by integrating the continuity equation (eq. 1). This is then used for each following step of the solution. This was solved using trapezoidal integration in Python

$$v(y) = -\int_0^y \frac{\partial u}{\partial x} dy \tag{14}$$

Now, this solution can be solved for various values of x.

Results

The following figure shows normalized velocity relative to wall distance at various downstream locations, up to $40 * L$ downstream of the initial location.

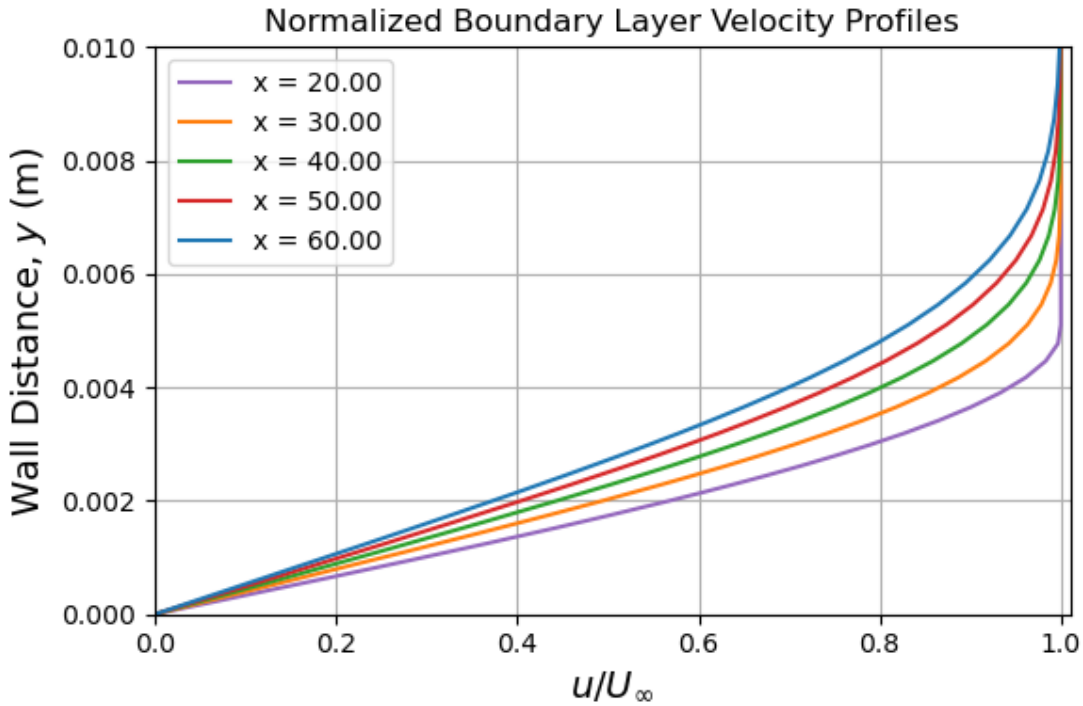

Figure 1: Numerical Solution at Various x-values

To determine the initial value of x, we must look at the Blasius Similarity Variable.

$$\eta_b = y\sqrt{\frac{U_\infty}{\nu x}} \tag{15}$$

For this, we can find x_0 by evaluating the Blasius similarity variable at the edge of the boundary layer, $u \approx 0.99U_\infty$, and $y = \delta$. The Blasius similarity variable at freestream can be approximated to $\eta_b = 5$ at this location [1].

$$x_0 = \frac{\delta^2 U_\infty}{25\nu} = \mathbf{20}\ (\boldsymbol{m}) \tag{16}$$

These velocity profiles can also be plotted against the Blasius Similarity Variable, in which convergence along the analytical solution becomes more apparent.

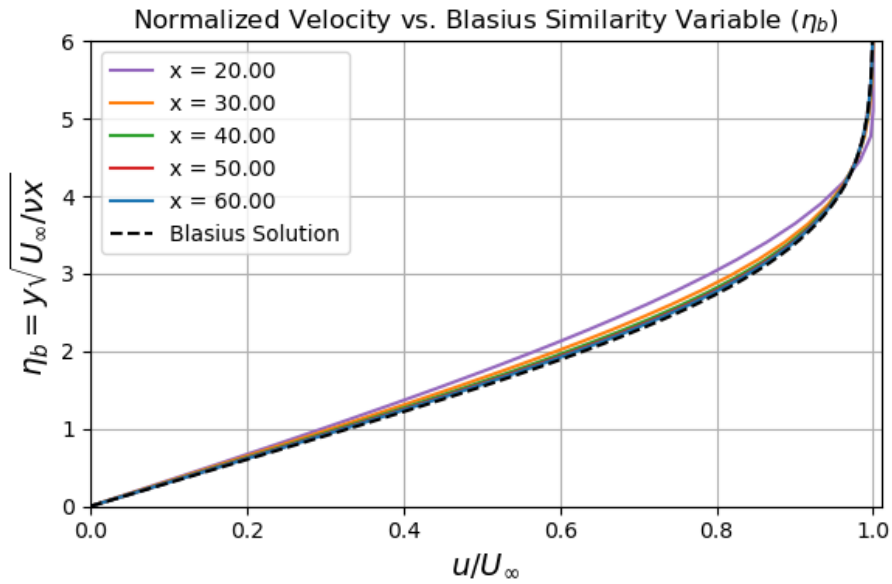

Figure 2: Comparison of Numerical Solutions and Blasius Solution

Momentum thickness and shape factor are both metrics that can help determine the convergence of the numerical solution. Momentum thickness, θ, is calculated with the following equation:

$$\theta = \int_0^\infty \frac{u}{U_\infty}\left(1 - \frac{u}{U_\infty}\right) dy \tag{17}$$

Shape factor is the ratio of displacement thickness, δ^*, to momentum thickness [2].

$$\delta^* = \int_0^\delta \left(1 - \frac{u}{U_\infty}\right) dy \tag{18}$$

$$H = \frac{\delta^*}{\theta} \tag{19}$$

The tables below summarize these parameters at various downstream locations, along with the parameters for the Blasius Solution.

Table 1: Boundary Layer Properties for Numerical Solutions

x (m)	Momentum Th., θ (m)	Displacement Th., δ^* (m)	Shape Factor, H
20	1.88e-03	6.97e-04	2.69
30	2.21e-03	8.33e-04	2.66
40	2.50e-03	9.52e-04	2.63
50	2.77e-03	1.06e-03	2.62
60	3.02e-03	1.16e-03	**2.61**

Table 2: Boundary Layer Properties for Blasius Solution (Dimensionless)

	Momentum Th., θ	Displacement Th., δ^*	Shape Factor, H
Blasius	0.664	1.72	**2.59**

Discussion

General

An expected boundary layer growth is observed in Figure 1. As x increases, the boundary layer pushes further from the wall.

Figure 2 gives proof that the numerical solution is approaching the analytical solution. As x increases, the initial piecewise assumption is forgotten, and the solution begins collapsing on the Blasius solution. At $x = 20$, there is a uniquely sharp kink, which is due to this piecewise approximation. This polynomial has a very abrupt transition to freestream velocity, whereas the Blasius solution has a more gradual transition.

It is also apparent that the numerical solution is converging on the Blasius solution based on the shape factor. The numerical shape factor is approaching 2.59, showing that the numerical solutions eventually will approach self-similarity, which is the expected outcome for a 2-D boundary layer problem.

Accuracy & Stability

The Crank-Nicolson scheme is an implicit scheme; therefore, it is unconditionally stable. In the boundary layer, we use a stretched grid, giving much higher near-wall resolution. As we move away from the wall, these partitions in the y-direction become larger. Having an unconditionally stable solver is particularly useful for this instance, as the solution will not blow up even when step sizes grow. The Crank-Nicolson scheme is also a 2nd order accurate solver, $O(\Delta x^2)$. This allows for a high-accuracy solution that remains stable once freestream velocity is achieved. Though C-N is unconditionally stable, we learned in Project 1 that it is still important to use an appropriate step size to ensure accurate solutions. This is achieved using the parameters given in the problem statement.

This project also used a 4th order central difference scheme for the interior spatial discretization in η. The use of a point, along with four neighboring points gives incredibly high accuracy, $O(\Delta y^4)$. Note that the boundary-adjacent points were solved with a 2nd order central difference method. While this offers slightly lower accuracy, it is still substantial. This higher order accuracy is apparent with the shape factor shown for $x = 60$ solution. A shape factor of 2.61 is less than 1 percent error when compared to the Blasius solution.

Appendix

[1] Ansys Innovation Courses. (2020, August). *Lesson 4: Flat plate (Blasius solution)* Ansys. https://innovationspace.ansys.com/courses/wp-content/uploads/2020/08/Lesson4-Flat-Plate-Blasius-Solution-Handout.pdf

[2] Aerospace Computing Lab. (2005, January). *Solutions of the Von Kármán integral momentum equation* [Lecture notes]. Stanford University. http://aero-comlab.stanford.edu/aa200b/lect_notes/lect8-9.pdf

Code

I had to restructure my Project 1 code more than expected. I believe now I have a much better solver. Now, it is much simpler to change the code to cater to a given problem.

I used Google Gemini to help with integration methods (trapezoid, cumulative_trapezoid) and in solving the Blasius integral with solve_bvp.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import lu_factor, lu_solve
import sympy as sp
from scipy.integrate import solve_bvp, trapezoid, cumulative_trapezoid
```

```
#given parameters
nu_inf = 1.5e-6 #freestream kinematic viscosity (m2/s)
u_inf = 30 #freestream vel (m/s)
L = 1 #(m)
delta = 0.005 #initial boundary layer thickness (m)

Ny = 150 #y partitions
y_max = 10*delta #max y
s = 10 #stretch factor

#estimate x0 for Blasius solution:
x0 = delta**2*u_inf/25/nu_inf #see report for derivation of this formula

x_tot = 40*L #independent var length
dx = 0.01 #x step size
Nx = int(x_tot/dx) #x partitions

#eta grid
eta = np.linspace(0, 1, Ny)
d_eta = eta[1] - eta[0]

#stretched y grid
y = y_max*(np.sinh(s*eta)/np.sinh(s))
```

```
#stretched y grid derivatives (solved for at end of code symbolically)
deta_dy = np.sinh(s)/(s*y_max*np.cosh(s*eta))
d2eta_dy2 = -(np.sinh(s)**2*np.sinh(s*eta))/(s*y_max**2*np.cosh(s*eta)**3)
```

```
#central diff, 4th order for interior points, 2nd order for boundary-
adjacent points
du_deta = np.zeros((Ny, Ny))
d2u_deta2 = np.zeros((Ny, Ny))

for i in range(1, Ny - 1):
    if i == 1 or i == Ny - 2:
        #2nd order cd, for boundary-adjacent points
        du_deta[i, i-1] = -1/(2*d_eta)
        du_deta[i, i+1] = 1/(2*d_eta)

        d2u_deta2[i, i-1] = 1/d_eta**2
        d2u_deta2[i, i] = -2/d_eta**2
        d2u_deta2[i, i+1] = 1/d_eta**2
    else:
        #4th order cd, for interior points
        du_deta[i, i-2] = 1/(12*d_eta)
        du_deta[i, i-1] = -8/(12*d_eta)
        du_deta[i, i+1] = 8/(12*d_eta)
        du_deta[i, i+2] = -1/(12*d_eta)

        d2u_deta2[i, i-2] = -1/(12*d_eta**2)
        d2u_deta2[i, i-1] = 16/(12*d_eta**2)
        d2u_deta2[i, i] = -30/(12*d_eta**2)
        d2u_deta2[i, i+1] = 16/(12*d_eta**2)
        d2u_deta2[i, i+2] = -1/(12*d_eta**2)
```

```
u = np.zeros(Ny) #init u
v = np.zeros(Ny) #init v

#at the initial location, assume that the velocity profile in the x
direction is given by:
for i in range(Ny):
    if y[i]/delta <= 1:
        u[i] = u_inf*(y[i]/delta*(3/2-1/2*((y[i]/delta)**2)))
    else:
        u[i] = u_inf

x_vals = [0,1000, 2000, 3000, 4000] #x steps ->x= 20 + n/100
u_store_numer = {0:u} #initialize u storing
v_store_numer = {0:v} #initialize v storing

for n in range(1, Nx + 1):

    #diffusion convection terms
    diff_conv = np.zeros((Ny, Ny))
    for i in range(1, Ny - 1):
        j = nu_inf*deta_dy[i]**2
        k = nu_inf*d2eta_dy2[i] - v[i]*deta_dy[i] #
```

```
        diff_conv[i, :] = (j*d2u_deta2[i, :] + k*du_deta[i, :]) #rearranged
PDE, du/dx = ... (see eq. 5 in report)

    #Au=b
    A = np.diag(u) -diff_conv*dx/2  #C-N: step n+1
    b = (np.diag(u) + diff_conv*dx/2)@u  #C-N: u*du/dx at step n

    b[0] = 0; #u=0 at wall
    b[-1] = u_inf #u=u_inf at freestream

    #top boundary
    A[-1, :] = 0;
    A[-1, -1] = 1;
    #bottom boundary
    A[0, :] = 0;
    A[0, 0] = 1;

    u_new = lu_solve((lu_factor(A)), b) #lu solve for u

    #update v using continuity eq, du/dx + dv/dy = 0
    du_dx = (u_new - u)/dx
    v_new = cumulative_trapezoid(-du_dx, x=y, initial=0)

    u = u_new
    v = v_new

    if n in x_vals:
        u_store_numer[n]=u
        v_store_numer[n]=v
```

```
plt.figure(figsize=(8, 6))
colors = ['tab:purple','tab:orange','tab:green','tab:red','tab:blue']
markevery = 2
for idx, n in enumerate(x_vals):
    x_now = x0 + n*dx
    plt.plot(u_store_numer[n]/u_inf, y, '-', color=colors[idx],
markersize=4, label=f'x = {x_now:.2f}')
plt.title('Normalized Boundary Layer Velocity Profiles', fontsize=12)
plt.xlabel(r'$u / U_{\infty}$', fontsize=14)
plt.ylabel(r'Wall Distance, $y$ (m)', fontsize=14)
plt.ylim(0, 0.01)
plt.xlim(0, 1.01)
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
```

```
#blasius solution for comparison
func = lambda x, y: [y[1], y[2], -0.5*y[0]*y[2]] #f''' = -0.5*f*f''
bcs = lambda ya, yb: [ya[0], ya[1], yb[1] - 1] #f(0)=0, f'(0)=0, f'(inf)=1

eta_span = np.linspace(0, 10, 100) #init range for eta
```

```
blasius_guess = np.zeros((3, eta_span.size)) #initial guess for blasius

blasius_eqn = solve_bvp(func, bcs, eta_span, blasius_guess) #solve boundary
value problem

eta_exact = blasius_eqn.x #eta values for blasius solution
u_exact_norm = blasius_eqn.y[1] #normalized to u_inf
```

```
#plot blasius normalized velocity profiles andcalc delta*, theta, and H at
each x location
d_stars = []#init lists
thetas = []
Hs = []

plt.figure(figsize=(6, 4))
colors = ['tab:purple','tab:orange','tab:green','tab:red','tab:blue']
markevery = 2
for idx, n in enumerate(x_vals):
    x_now = x0 + n*dx
    #---------plotting---------------------------
    eta_now = y*np.sqrt(u_inf/(nu_inf*x_now))
    plt.plot(u_store_numer[n]/u_inf, eta_now, '-', color=colors[idx],
markersize=4, label=f'x = {x_now:.2f}')
    #---------------------------------------------

    #---------calculations for dstar, theta, H----
    d_star = trapezoid(1-u_store_numer[n]/u_inf, x=y) #disp
thickness=integral of (1-u/u_inf) dy (m)
    theta = trapezoid(u_store_numer[n]/u_inf*(1-u_store_numer[n]/u_inf),
x=y) #momentum thickness=integral of (u/u_inf)(1-u/u_inf) dy (m)
    H = d_star/theta #dimensionless ratio of disp to momentum thickness

    #store vals
    d_stars.append(d_star)
    thetas.append(theta)
    Hs.append(H)
    print(f"for x = {x_now:.0f}: dstar = {d_star:.6e}, theta = {theta:.6e},
H = {H:.3f}")
    #---------------------------------------------

plt.plot(u_exact_norm, eta_exact, '--', color='k',label=f'Blasius Solution')
plt.title(r'Normalized Velocity vs. Blasius Similarity Variable ($\eta_b$)',
fontsize=12)
plt.xlabel(r'$u / U_{\infty}$', fontsize=14)
plt.ylabel(r'$\eta_b = y \sqrt{U_{\infty} / \nu x}$', fontsize=14)
plt.ylim(0, 6)
plt.xlim(0, 1.01)
plt.legend(loc='upper left')
plt.grid()
plt.tight_layout()
plt.show()
```